Assignment - 1

Module 13 – Python Fundamentals

- **Introduction to Python**

1. Introduction to Python and its Features (simple, high-level, interpreted language).

- ⇨ Python is a simple and popular programming language and created by Guido van Rossum.
- ⇨ It is easy to learn and widely used for web development, data analysis and automation.
- ⇨ Key Features :-
- **Simple Language :-** Easy to read and write
- **High-Level :-** No need to manage memory manually
- **Interpreted :-** Runs code line by line
- **Platform Independent :-** Works on Windows, Linux and MacOS.
- **Dynamically Typed :-** No need to declare variable types
- **Large-Library :-** Many Built in tools available

2. History and evolution of Python.

- ⇨ **History Of Python :-**
  Python was created Guido van Rossum in late 1980s and officially released in 1991. It was designed to be simple, readable and easy to use.

- ⇨ **Evolution of Python :-**
  - **Python 1.x (1991) :-** First Version with basic features like variables loops and functions.
  - **Python 2.x(2000) :-** Added New Features like better memory management and support for more libraries. It became very popular but is now discontinued.
  - **Python 3.x(2008-Present) :-** Improved versions with better syntax, performance and modern features.it is current and most widely used version.

3. Advantages of using Python over other programming languages.

- ⇨ There are below the advantages of python programming :-
- ✓ **Easy to Learn and Use :-** Simple syntax makes it beginner friendly
- ✓ **Readable Code :-** Codes looks clean and easy to understand.
- ✓ **High Productivity :-** Fewer lines of code compared to many other languages.

- ✓ **Large Standard Library :-** Comes with many built in modules for different tasks
- ✓ **Cross-Platform :-** Runs on Windows, Linux and MacOS without changes.
- ✓ **Versatile :-** Used in web development, data science, AI, Automation, etc..
- ✓ **Strong community support :-** Large community provides tutorials, tools and help.

4. Installing Python and setting up the development environment (Anaconda, PyCharm, or VS Code).

⇨ **Installing Python :-**

- Go to the official website: python.org
- Download the latest version
- Run the installer
- ✔ Important: Check **"Add Python to PATH"** before installing

⇨ **Set Up Development Environment :-**

➢ **VS Code :-**

- Download from code.visualstudio.com
- Install and open it
- Install **Python extension**
- Select Python interpreter
- Lightweight and flexible

5. Understanding Python's PEP 8 guidelines.

⇨ PEP Stands for Python Enhancement Proposals.
There are several PEP 8 guidelines

- **Indentations & Spacing :-**
  Uses 4 spaces per indentation level (no tabs)
  Limit lines to 79 characters.
  Use blank lines to separate functions and classes.

- **Naming Conventions :-**

  Variable & functions :- Uses Snake_case (Ex:- name = "ravi")

  Classes :- Uses PascalCase (Ex:- name = "BankAccount")

  Constants :- Uses UPPERCASE (Ex:- MAX_USERS = 100)

- **Imports :-**
  Put imports at the top of the file.

Group them in this order :-

- ✓ Standard Library
- ✓ Third-Party Libraries
- ✓ Local Imports

- **Whitespaces Rules :-**

  Avoid Unnecessary Spaces

  Ex :-

  X = 5 (right)

  X=5 (wrong)

  Use spaces around the operator :-

  Ex:- y = x + 5

- **Functions & Arguments :-**

  Keeps functions small and focused.

  Use descriptive names

  Add default arguments where useful

- **Docstrings :-**

  Use docstrings to explain what functions/classes does.

  Ex:-

  ```
  def add(a, b):
    """Return the sum of two numbers."""
    return a + b
  ```

  ==> here,(“”“) is a docstrings

- **Comments**
- **Comparisons & Booleans :-**

  Use clean Comparisons

  Ex:-

  ```
  if value is None:     # correct
  if value == None:     # wrong
  ```

  Avoid Unnecessary Boolean comparisons

  Ex:-

  ```
  if is_valid:          # correct
  if is_valid == True:   # wrong
  ```

- **Classes Best Practices :-**
  Keep methods organized
  Use _Init_ Properly.

6. Understanding data types: integers, floats, strings, lists, tuples, dictionaries, sets.

⇨ Here's below the basic understanding the data types :-

Data Type	Description	Example	Mutable?	Ordered?	Notes/Uses
Integer(int)	Whole Number, positive and Negative	42, -7, 0	No	Yes	Can be used in Arithmetic Operations.
Float(float)	Decimal Numbers(real numbers)	3.14, 2.0	No	Yes	Precision may be limited, but good for measurement and calculations.
String(str)	Sequence of Characters(text)	"Hello", '123'	No	Yes	Can be indexed or sliced and supports many string methods
List(list)	Ordered collections of items (any data type)	[1,2,4], ["a",1,3.5]	Yes	Yes	Can add, remove and modify items.
Tuple(tuple)	Ordered collections of items (any data type)	(1,2,3), ("a",4)	No	Yes	Immutable; good for fixed collections
Dictionary (dict)	Key-value pairs fast lookup	{"name" : "yash", "age" : 25}	Yes	No	Key must be unique and immutable, values can be any type
Set (set)	Unordered collection of unique items	{1,2,3}, {"a","b"}	Yes	No	No duplicate allowed. Useful for membership tests and math ops.

7. Python variables and memory allocation.

⇨ **Variable** :- it is a simply name that points to an object in memory.
It doesn't the store the value it's self.
Ex:- **x = 10 ,** x is a variable and 10 is an object

Memory Allocation :-
Python Automatically stores the value in memory(heap)
You don't manage memory manually.

Garbage Collection :-
Python Automatically deletes unused objects.
No manual memory freeing needed.

8. Python operators: arithmetic, comparison, logical, bitwise.

⇨ There are below the operators :-

➢ **Arithmetic Operator :-**

Operator	Example	Description
+	Print(5 + 2) = 7	Addition
-	Print(5 – 2) = 3	Subtraction
*	Print(5 * 2) = 10	Multiplication
/	Print(5 / 2) = 2.5	Division (float)
//	Print(5//2) = 2	Floor division
%	Print(5%2) = 1	Modulus (remainder)
**	Print(5**2) = 25	Exponent

➢ **Comparison Operator :-**

Operator	Example	Description
==	5 == 2 -> False	Equal to
!=	5 != 2 -> True	Not equal
<	5 < 2 -> False	Less than
>	5 > 2 -> True	Greater than
<=	5 <= 2 -> False	Less or equal
>=	5 >= 5 -> True	Greater or equal

➢ **Logical Operator :-**

Operator	Example	Description
And	True and False -> False	True if both are true
Or	True or False -> True	True if one is true
Not	Not False -> True	Reverses Results

- **Bitwise Operator :-**

Operator	Example	Description
&	5 & 3 -> 1	AND
|	5 | 3 -> 7	OR
^	5 ^ 3 -> 6	XOR
~	~ 5 -> -6	NOT
<<	5 << 1 -> 10	Left shift
>>	5 >> 1 -> 2	Right shift

9. Introduction to conditional statements: if, else, elif.

⇨ Below the conditional statements

If statement :- if statement checks the conditions. If condition is true then run the code.

Example :-

```
Age = 18
If age >=18:
        Print("Eligible for voting")
```

Else Statement :- else statement runs when if conditions false.

Example :-

```
Age = 16
If age >=18:
        Print("Eligible for voting")
Else:
        Print("not eligible for voting")
```

Elif Statement :- elif conditions checks the multiple conditions

Example :-

```
marks = int(input("Enter the Marks = "))

if marks>100:
   print("Invalid Marks Obtained!...")

if marks > 90 and marks <= 100:
   print("Grade A")
elif marks > 70 and marks <=90:
   print("Grade B")
elif marks > 50 and marks <=70:
   print("Grade C")
elif marks > 30 and marks <=50:
   print("Grade D")
elif marks > 0 and marks <=34:
   print("Grade F")
else:
   print("Something Went wrong")
```

10. Nested if-else conditions.

⇨ Nested if-else statement means one if-else statement inside another if or else block.

Example :-

```
num = int(input("Enter the Number = "))
if num>0:
   if num % 2 == 0:
      print("Positive and Even Number")
   else:
      print("Negative and Odd Number")
else:
   print("Number is Negative")
```

11. Introduction to for and while loops.

⇨ Below the introduction of for and while loops

➢ **For Loop :-**

It is used when you want to iterate over a sequence like a list, string, tuple or range.

Syntax :-

```
for variable in sequence:
        #code block
```

Example :-

```
for I in range(10):
        print(i);
```

- **While Loop :-**

While loops keeps running as long as a condition is true.

Syntax :-

```
While condition:
        #code block
```

Example :-

```
i=20;
while i<=30
      print(i)
      i=i+1
```

12. How loops work in Python.

⇨ Loops in python are a way to repeat a block of code automatically, instead of writing the same code multiple times.
A loop has three main parts :-
Start point -> when the loop begins.
Condition or sequence -> decides how long it runs.
Update -> changes something each time to avoid infinite looping.

13. Using loops with collections (lists, tuples, etc.).

⇨ Below the some examples of using loops with collections

1. **List :-**

```
Fruits = ['apple','banana','cherry']
For I in fruits:
        Print(I)
```

2. **Tuple:-**

```
Car = ('BMW','PAGANI','AGERA R')
For car in Car:
        Print(car)
```

3. **Set:-**

```
Num = {1,2,3}
For num in Num:
        Print(num)
```

4. **Dictionary:-**

```
Person = {name:"yash", age: 25}
For key, value in person.items():
        Print(key,value)
```

5. **Index :-**

```
for i, val in enumerate(fruits):
  print(i, val)
```

14. Understanding the role of break, continue, and pass in Python loops.

⇨ Below the explanation :-

- **Break Statement :-** stop the loop completely.

  Example :-

```
For I in range(5):
    If I == 3:
        Break
    Print(I)
```

- **Continue Statement :-** skip current iterations, goes to the next

  Example:-

```
For I in range(5):
    If I == 3:
        continue
    Print(I)
```

- **Pass Statement :-** Does nothing

  Example :-

```
For I in range(5):
    If I == 3:
        pass
    Print(I)
```

15. Understanding how to access and manipulate strings.

- Understanding strings in python is a core skill – you'll use it everywhere , from simple input handling to building full applications.
- **What Is String?**

  a string is just a simple text:

  name = 'yash'

  Accessing characters in string:

  Strings are indexed (like list), starting from 0

```
Name = 'yash'
Print(Name[0])
Print(Name[1])
```

  And also slice the strings.

16. Basic operations: concatenation, repetition, string methods (upper(), lower(), etc.).

- below the operations :-

  String Concatenations:-(+)

  Used to join two sequences (like string, list, tuple)

  Example :-

```
Print("Hello")
Print("World")
Print(a + b)
```

  Output:-

  HelloWorld

  String Repetition :-(use join and *)

  Used to repeat sequence multiple times

Example:-
A = ‘Hii’
Print(A*3)

Output:-
HiiHiiHii

Example 2:-
B = “Hel”
Print(B.join(“Hel”))

String Methods :-

String Methods	Description
Len()	Return the length of the string
Lower()	Convert all characters into small letter
Casefold()	Convert other languages symbol(like german language(zetta))
Upper()	Convert all characters into capital letter
Capitalize()	First character of strings to uppercase.
Title()	Convert the first character of each word strings to uppercase
Strip()	Remove spaces(at the beginning) and at the end
Replace()	Replace a substring with another string
Find()	Searches for a substring and return the index of first
Startswith()	Check if the string ends with specified substring
Endswith()	Check if the string ends with specified substring
Split()	Split the string into a list of substring
Join()	Joins list string into a single string, which specified separator between each element

Isalpha()	Return true if all characters in strings are alphabetical
Isdigit()	Return true if all characters in strings are numerical
Isalnum()	Return true if all characters in strings are alphanumerical
Zfill()	Pads string with zeros(0) on left string until reaches specified length
Center()	It is used to center-align string with in specified width by adding padding character to both sides.

17. String slicing.

⇨ String slicing in python is a way to extract portion of a string using a simple and powerful syntax.

Syntax:-

String[start: stop: step]

Where,

Start – index to begin

Stop – index to stop

Step – how many characters to skip

Example:-

Text = “Hello World”

Print(Text[0 : 5])

18. Built-in methods for strings, lists, etc.

⇨ Below the list built in methods:-

String Methods :-

String Methods	**Description**
Len()	Return the length of the string
Lower()	Convert all characters into small letter

Casefold()	Convert other languages symbol(like german language(zetta))
Upper()	Convert all characters into capital letter
Capitalize()	First character of strings to uppercase.
Title()	Convert the first character of each word strings to uppercase
Strip()	Remove spaces(at the beginning) and at the end
Replace()	Replace a substring with another string
Find()	Searches for a substring and return the index of first
Startswith()	Check if the string ends with specified substring
Endswith()	Check if the string ends with specified substring
Split()	Split the string into a list of substring
Join()	Joins list string into a single string, which specified separator between each element
Isalpha()	Return true if all characters in strings are alphabetical
Isdigit()	Return true if all characters in strings are numerical
Isalnum()	Return true if all characters in strings are alphanumerical
Zfill()	Pads string with zeros(0) on left string until reaches specified length
Center()	It is used to center-align string with in specified width by adding padding character to both sides.

List Methods :-

List Methods	**Description**
Append()	Adds element at the end of the list.
Clear()	Remove all element from list

Copy()	Return a copy of the list
Count()	Return the number of element with the specified value
Extend()	Add the element of list, to the end of current list
Index()	Return the index of the first element with specified value
Insert()	Add the element at the specific value
Pop()	Remove the element at the specified position
Remove()	Remove the item with specified value
Reverse()	Reverse the order of list
Sort()	Sort the result.

19. Writing readable and maintainable code.

⇨ Below the guideline of readable and maintainable code:-

1. **Follow the standards (PEP 8) :-** Indentation, spacing and casing
2. **Use meaningful name**
3. **Keep functions small and descriptive name**
4. **Use type hints :-** tell python what data type to expect. It prevents bugs before they happen
5. **Be pythonic :-** use built-in tools like list comprehensions and f-strings to keep code concise.
6. **Document “Why, ”Not, “What” :-** Don't state the obvious. Explain the logic.